

Homework 1

January 28, 2019

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from scipy.stats import norm
import sklearn.linear_model
```

1 Error measures and prediction.

(a) We choose estimator I, which has lower mean squared prediction error.

```
In [2]: v1 = stats.expon.rvs(size=1000)
EI = np.mean((v1 - 1)**2)
EII = np.mean((v1 - np.log(2))**2)
print(f"Error I = {EI}")
print(f"Error II = {EII}")
```

Error I = 0.9394766428533355

Error II = 1.0026029437530273

(b) We choose estimator II, which has lower mean squared prediction error.

```
In [3]: v1 = stats.expon.rvs(size=1000)
EI = np.mean(np.abs(v1 - 1))
EII = np.mean(np.abs(v1 - np.log(2)))
print(f"Error I = {EI}")
print(f"Error II = {EII}")
```

Error I = 0.7586754144991771

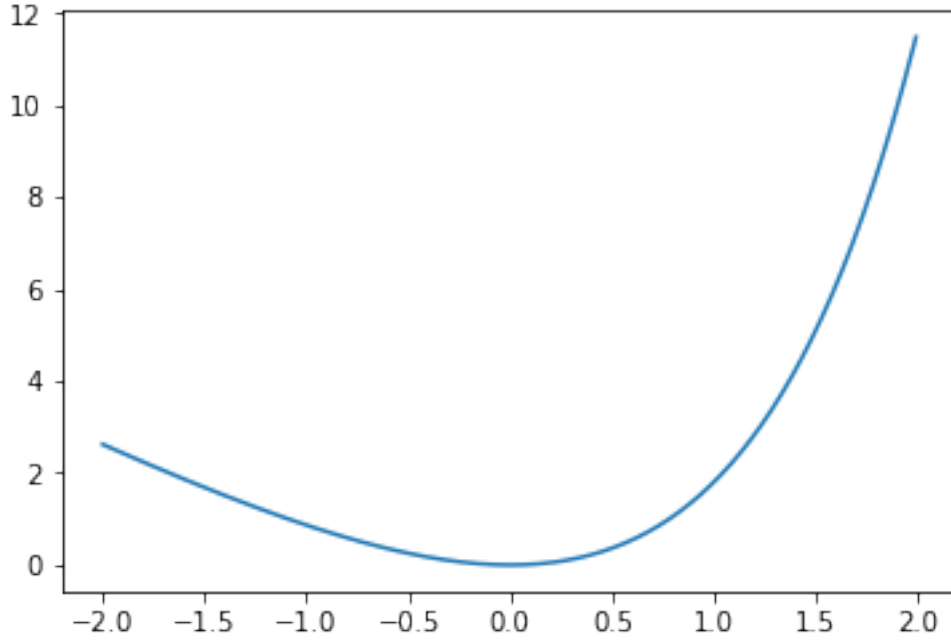
Error II = 0.732369881015371

2 Asymmetric loss and the regression function.

(a) The loss function goes down first and until some point it goes up again. I want to use a loss function like this because it has a minimum where a potential best estimator come from.

```
In [4]: z = np.arange(-2,2,0.01)
a = 1.1
b = 2
loss = b*(np.exp(a*z) - a*z-1)
plt.plot(z, loss)
```

```
Out[4]: [<matplotlib.lines.Line2D at 0x1a14203c50>]
```



(b)

$$\mathbb{E}(\mathcal{L}(Y, f(X))) = \mathbb{E} \left(\mathbb{E} \left(b(e^{a(Y-f(X))} - a(Y-f(X)) - 1) | X \right) \right) \quad (1)$$

$$= \mathbb{E} \left(b(e^{-af(X)} \mathbb{E}(e^{aY} | X) - a\mathbb{E}(Y|X) + af(X) - 1) \right) \quad (2)$$

$$\frac{\partial \mathbb{E}(\mathcal{L}(Y, f(X)))}{\partial f(X)} = b(-ae^{-af(X)} \mathbb{E}(e^{aY} | X) + a) = 0 \quad (3)$$

$$f(x) = \frac{1}{a} \log(\mathbb{E}(e^{aY} | X = x)) \quad (4)$$

(c)

$$\mathbb{E}(e^{aY} | X = x) = e^{\beta ax + \frac{1}{2} \sigma^2 a^2} \quad (5)$$

$$f(x) = \frac{1}{a} \log e^{\beta ax + \frac{1}{2} \sigma^2 a^2} \quad (6)$$

$$= \beta x + \frac{1}{2} a \sigma^2 \quad (7)$$

(d) Average loss of the conditional expectation: 14.46 Average loss of my method: 4.28 My average loss is lower than that of conditional mean. This is because the loss function is not symmetric, in this case using the conditional expectation will lead to greater loss.

```
In [6]: #Set some parameters
        beta = 0.5
        b = 2
        sigma = 2
        a = 1

        #Define the loss function, where  $z = y - \hat{y}$ 
        def loss(z):
            return b*(np.exp(a*z)-a*z-1)

        #Estimation functions
        #Estimation using the conditional expectation of  $Y|X$ 
        def f_condexp(x):
            return beta*x

        #TODO: Put your function in here. You can reference a,b,sigma, and it will just pull
        # the outside namespace
        def f_yours(x):
            return beta*x+0.5*(a)*(sigma**2)

        #Simulation to see how you do
        reps = 1000

        #Just generate the X variables normally. We don't really care
        x = norm.rvs(size=reps, loc=0, scale=1)

        #Generate the Y variables from our normal model
        y = norm.rvs(size=reps, loc=x*beta, scale=sigma)

        #Calculate the fitted values for each method
        yhat_condexp = np.apply_along_axis(f_condexp, 0, x)
        yhat_yours = np.apply_along_axis(f_yours, 0, x)

        #Compute the losses
        condexp_losses = np.apply_along_axis(loss, 0, y-yhat_condexp)
        your_losses = np.apply_along_axis(loss, 0, y-yhat_yours)

        print("Average loss of the conditional expectation:",
              round(np.mean(condexp_losses),2))

        print("Average loss of your method:",
              round(np.mean(your_losses),2))
```

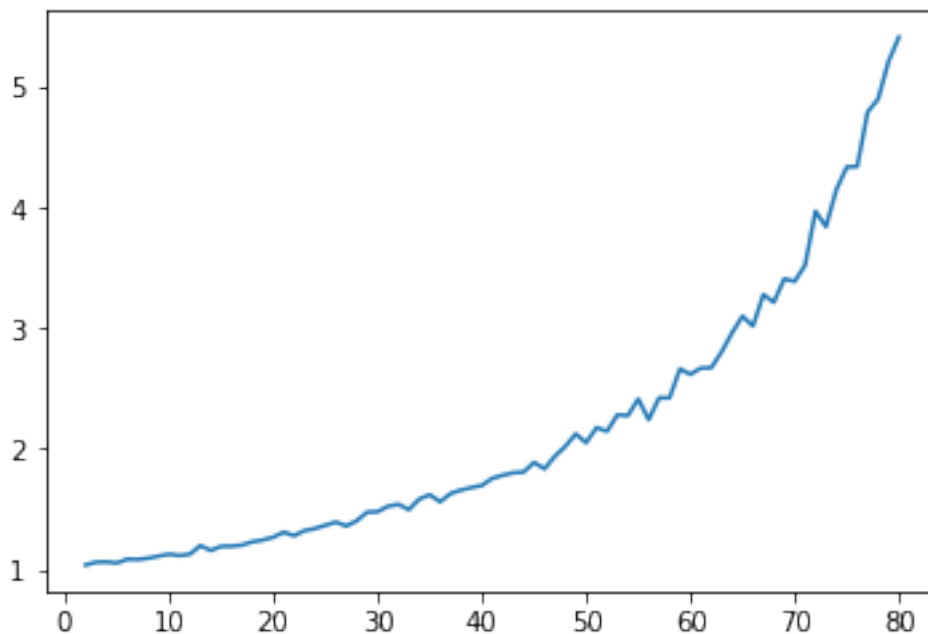
Average loss of the conditional expectation: 14.46

Average loss of your method: 4.28

3 What happens to linear regression in high dimensions?

As dimension p increases, mean squared prediction error increases.

```
In [7]: def RV_generator(p):  
    n = 100  
    X = np.random.randn(n, p)  
    Y = X[:, 0] * 4 + np.random.randn(n)  
    return X, Y  
  
avg_mse = []  
for p in range(2,81):  
    mse = 0  
    for i in range(100):  
        X,Y = RV_generator(p)  
        X_test,Y_test = RV_generator(p)  
        model = sklearn.linear_model.LinearRegression()  
        model.fit(X,Y)  
        Y_pred = model.predict(X_test)  
        mse += sklearn.metrics.mean_squared_error(Y_test, Y_pred) / 100  
    avg_mse.append(mse)  
  
plt.plot(range(2,81),avg_mse)  
plt.show()
```



4. (a) We want to show

$$\|\tilde{y} - \tilde{x}\beta\|_2^2 = \|y - x\beta\|_2^2 + \lambda \|\beta\|_2^2$$

From LHS, We have: $(\tilde{y}^T - \beta^T \tilde{x}^T)(\tilde{y} - \tilde{x}\beta)$

$$= \tilde{y}^T y - \tilde{y}^T \tilde{x} \beta - \beta^T \tilde{x}^T \tilde{y} + \beta^T \tilde{x}^T \tilde{x} \beta$$

$$\tilde{y} = \begin{bmatrix} y \\ 0 \end{bmatrix}, \tilde{x} = \begin{bmatrix} x \\ \lambda \mathbf{1} \end{bmatrix}, = y^T y - y^T x \beta - \beta^T x^T y + \beta^T x^T x \beta + \lambda \beta^T \beta$$

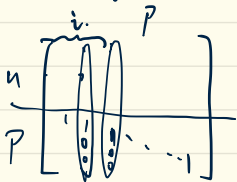
From RHS, We have: $(y^T - \beta^T x^T)(y - x\beta) + \lambda \beta^T \beta$

$$= y^T y - y^T x \beta - \beta^T x^T y + \beta^T x^T x \beta + \lambda \beta^T \beta$$

$\therefore \text{LHS} = \text{RHS}$ $\square$

(b) Proof by Contradiction

Suppose $\tilde{x}$ does not have full rank. i.e. columns are not linear independent



Pick the i^{th} column, it can be written as the linear combination of the remaining columns. $x_i = \sum_{j \neq i} a_j x_j$.

However, the $(n+i)^{\text{th}}$ element in x_i is $\sqrt{\lambda}$, in $x_j (j \neq i)$ is 0.

linear combination of zeros cannot give $\sqrt{\lambda} \neq 0$. $\Rightarrow$ Contradiction

$\therefore \tilde{x}$ has full rank. $\square$

(c) By part (a), $\beta_{\text{ridge}} = (\tilde{x}^T \tilde{x})^{-1} \tilde{x}^T \tilde{y}$

$$= (\tilde{x}^T x + \lambda \mathbf{I})^{-1} \tilde{x}^T y.$$

$\therefore a^T \beta_{\text{ridge}} = a^T (\tilde{x}^T \tilde{x})^{-1} \tilde{x}^T \tilde{y}$, where $\tilde{y} = \begin{bmatrix} y \\ 0 \end{bmatrix} \in \mathbb{R}^{(n+p)}$

let $a^T (\tilde{x}^T \tilde{x})^{-1} \tilde{x} = A$. $\in \mathbb{R}^{1 \times (n+p)}$

$\Rightarrow a^T \beta_{\text{ridge}} = A \tilde{y}$ is simply the linear combination of first n elements in A and y . $\square$